Spring 2026 CS 2210
Project Description

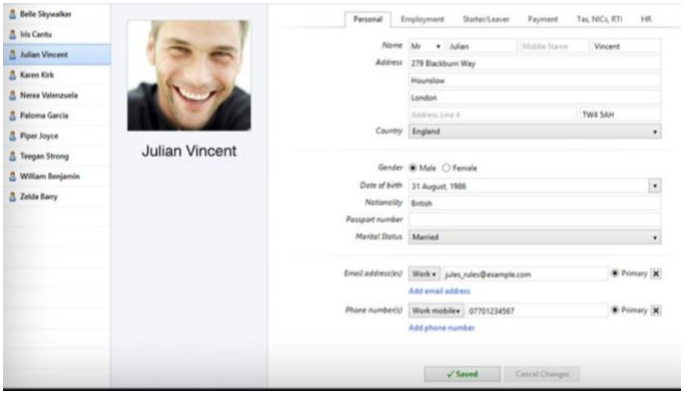
Requirements

1) You **MUST** form teams of **two** or a *maximum* of **three** students. *So far (01/08/2026) there are 12 students in the class, that means that we should have four teams of 3 students.* Use the "Teams information" page on Canvas to know the teams.
2) *You **must** join a team!* (See the documents I have posted about the advantages of working in a team for software development.)
3) Teams can be formed with students on the in-person section, students from the remote section, or a mix of both.
   a) Once teams are formed, **they cannot change**. If problems arise within the teams, then the team can split. However, each newly formed team needs to continue working on the same project independently. Each newly formed team will be graded independently. If a team decides to split, then I need to receive a confirmation from ***all*** *members*.
4) You will use the Java elements from the course.
5) You need to finish the tasks stated in the **four stages** of the project. NOTE that the due dates of the four stages are different.
6) The team needs to create a ***public GitHub*** repository for the project.
   a) Inside the GitHub repository, create **four folders, one for each stage** of the project.
   b) Be careful with the modification dates!
   c) Each folder must contain only the deliverables for that stage (i.e., source code, text files, diagrams).
7) Each team must select **one** of the following projects:
   (I will use entities and classes interchangeably for some parts of the requirements.)
   a) An **airline *management system***
   The system must cover various functionalities from managing the different entities interacting in the system.
   The following *entities* are mandatory: **Passenger, Staff, Flight, Arline, Schedule, Booking.**
   You **must** enrich this system with **additional entities**. All entities must implement *CRUD tasks (defined below).*

   b) A ***conference management system***
   The system must cover various functionalities from managing the different entities interacting in the system.
   The following *entities* are mandatory: **Attendee, Speaker, Room, Schedule, Conference, Organizer.**

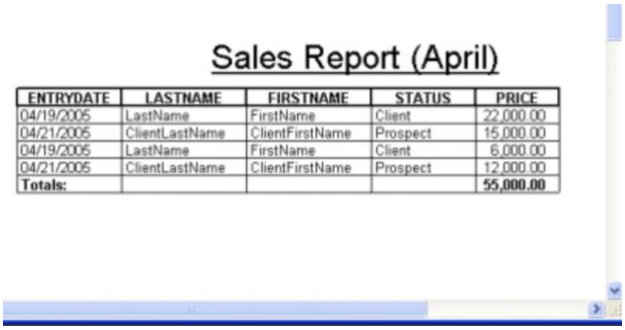
You **must** enrich this system with **additional entities**. All entities must implement *CRUD tasks (defined below).*

8) After selecting one of these systems, each team decides the project scope. However, there are some points the project **must include:**
   a) Clear definition of the project. The projects' scope needs to be **fair for a *semester-long* project**. This is, a project not too simple (e.g., one or two entities) and not too ambitious (e.g., the ERP of a company!) I will evaluate the scope of your problem, and I will make suggestions and changes during the second stage.
   b) You need to use a *UML software* for creating your UML class diagrams (e.g., https://www.lucidchart.com/pages/es/ejemplos/diagrama-uml). These diagrams need to be updated, if necessary, during the project.
   c) For your system, you **must** define **some** classes that use **inheritance.**
   d) The system needs to use **persistent information**. This is, Load and Save information in **local files** (e.g., .csv, .txt, .dat) (this is for stage 4) [I will briefly cover this topic with a couple of examples. Doing your own research is expected!]
   e) The system must allow the user to **Create**, **Read**, **Update**, and **Delete** (*CRUD*) the information of the main objects. For example, create new employees, generate a new sale, modify the information about a customer, delete an old item for the catalog, etc.
   f) The project needs to have **catalogs** of objects (e.g., a catalog of clients, a catalog of items, a catalog of employees, etc.). A catalog list all the elements of a given class and its features. Apply CRUD to these features. Likewise, a ***search method* is expected** (search applies for Stage 4 - GUI)



(This is *just* an example to illustrate a catalog of employees)

g) The system needs to "print" on screen at least two reports (e.g., sales, inventory, payroll) (one report for Stage 3, two reports for Stage 4)

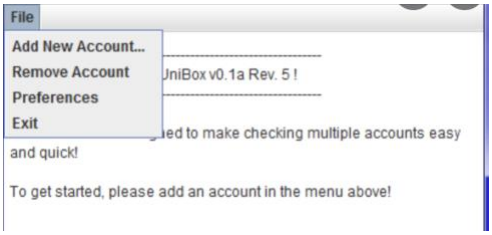


Sales Report (April)

ENTRYDATE	LASTNAME	FIRSTNAME	STATUS	PRICE
04/19/2005	LastName	FirstName	Client	22,000.00
04/21/2005	ClientLastName	ClientFirstName	Prospect	15,000.00
04/19/2005	LastName	FirstName	Client	6,000.00
04/21/2005	ClientLastName	ClientFirstName	Prospect	12,000.00
Totals:				**55,000.00**

(This is *just* an example to illustrate a report)

h) The system needs to have a menu to navigate through the different sections



(This is *just* an example to illustrate a menu. You can design your menus in a different way.)

i) The **GUI interface** (in Stage 4) must use **multiple windows** (e.g., open new windows or use a tabbed panel)

6. You must create an **executable** *Jar* file. Your program must run just making double click on the executable file (in Stage 4)

7. For Stage 3, **if** your program **cannot be compiled** (i.e., compilation errors) or if it **cannot be executed** (i.e., compiles but it does not work), the grade will be **zero** (also see the grading rubric for programming assignments). So, make sure your submission works!

Plagiarism: If I detect any plagiarism of projects in Stage 3, then, Stage 3, and Stage 4, **both parts**, will have a grade of zero (because Stage 4 depends on Stage 3). **It has happened! Don't risk your grade.**

Grading rubric: I will use a grading rubric for the code. Check each requirement.

Teamwork grading form: I will use the peer grading form to assign the grades for each stage.
Each team member **MUST** upload their peer grading form.

The following is the description for each individual state:

Stage 1 (100 points)

For this stage you need to do the following:

a) Define your team and type of project (**100 points**)
   i) Create a **single .PDF file** in your project stage's folder (in GitHub) with:
      (1) the type of system you have selected. (**10 points**)

      (2) description of your project (i.e., what are you planning to have in your system). Your description should consider **the required entities** and any additional entities you suggest(**80 points**)

   ii) There are six required classes. There are at most three members per team. Each team member must be responsible of at two of these classes for all stages. If the team has less than 3 members, the classes must be split evenly.
   Your project's folder must contain a **TXT** file **per team member** (i.e., *one file per team member*). The name of the file must be **lastname_firstname.txt** (i.e., your name). The text file must contain the names of your selected classes.
   Important: Don't create files for your teammates but make sure they create their own! (I can check on GitHub who created each file) (**10 points**)

Submission:

a) Submit in Canvas the GitHub **link** to access the corresponding folder (be aware of the modification date!)
b) Submit in Canvas your peer grading form.

Helpful links:

- Working with GitHub
- Inviting collaborators to a personal repository
- How to Upload Files to GitHub Through Terminal
- Uploading Files to GitHub
- How to Add Files to a GitHub Repo You Don't Own

Due date: Sunday, February 8 (10:00 p.m.) [tentative]

Stage 2 (100 points)

In this stage, you need to do the following (based on my recommendations from the previous stage)

a) Design the different classes and relationships (i.e., inheritance you have defined in the previous stage). Your project's folder must contain a **single PDF** file with your basic UML diagrams representing your system's classes (i.e., attributes, methods) and their relationships. (**80 points**)

b) Your project's folder must contain a **TXT** file **per team member** (i.e., *one file per team member*)with the job done by that team member. (Everybody needs to work on the GitHub repository. I'll check the commits.) **Don't create files for your teammates but make sure they create their own!** (I can check on GitHub who created each file) (**10 points**)

c) One team member needs to be the project leader for this stage and needs to **meet me** before the due date for reviewing the work done.

The meeting needs to be done **before** the due date to present the progress of the work. If the team leader does not meet with me, the whole team will get the deduction. The team members must hold the team leader accountable. The team leader will receive an additional deduction as they have the responsibility to represent the team (10 points).

Important: Each team member is fully responsible for completing the implementation of the classes they selected in Stage 1. If a team member does not complete their assigned work, they will receive a grade of **zero** for this stage.

However, the team's primary goal is to ensure the overall system remains functional. In the event that a team member fails to deliver their portion, the rest of the team is expected to redistribute tasks or create alternative solutions to address the missing functionality.

It is critical to document such situations clearly. If a team member does not fulfill their responsibilities, this must be noted in the peer grading form, detailing how their absence impacted the project and how the team mitigated it. While teamwork is encouraged, accountability is paramount. No free rides!

Submission:

a) Submit in Canvas the GitHub **link** to access the corresponding folder (be aware of the modification date!)

b) Submit in Canvas your peer grading form.

Due date: Sunday, March 8 (10:00 p.m.) [tentative]

Stage 3 (100 points)

In this stage, you need to do the following:

a) Create your **complete system** as a ***command-line*** application (**55 points total**)
   i) Your system must have a **main Java file** (called Main.java) to run your software. and separate java files, one for each class (**10 points**)
   ii) You **must** use **inheritance** for in any of your defined classes in this stage. **Your system needs to have the required classes** and any other class needed. (**10 points**)
   iii) Your system must present a **menu with submenus**. The user uses that menu to navigate through the different sections. Your program should end until the user selects an option to quit.(**10 points**)
   iv) Your system **must show** on screen at least **one** report (e.g., sales, customers)(**10 points**)
   v) Your system, as an initial task, **must** create **initial dummy information** for the objects. For example, some customers, some transactions. (**10 points**)
   vi) The system **must** allow the user to **create**, **read**, **modify**, and **delete** (CRUD) information of objects for your classes. For example, client information, employee information, patient information, item information. (**15 points**)

b) Upload to **GitHub** your source files into the corresponding folder for part 3 (**5 points**)
c) Upload to GitHub a **readme.txt** file with the instructions on **how to compile and how to run** your program (this is important!) (**5 points**)
d) Upload to GitHub a **single PDF** file with the **modifications** to your basic UML diagrams. The diagrams should show your program's classes (i.e., attributes, methods) and their relationships that you use for the first part of the project (**5 points**)
e) Your project's folder must contain a **TXT file per team member** with the job done by that team member. Everybody needs to work on the GitHub repository. I'll check the commits. (**5 points**)
f) Upload to GitHub a **single PDF** document a brief **user guide** on how to use your system and the most important sections (use screenshots of your system running and descriptions of each image)(**10 points**)

g) Upload a **PDF** with the screenshot of the files in your stage folder (**5** points).
h) One team member needs to be the project leader for this stage (different team member from the previous stage) and needs to meet me before the due date for reviewing the work done. The meeting needs to be done **before** the due date to present the progress of the work. If the team leader does not meet with me, the whole team will get the deduction. The team members must hold the team leader accountable. The team leader will receive an additional deduction as they have the responsibility to represent the team (**10** points).

Important: Each team member is fully responsible for completing the implementation of the classes they selected in Stage 1. If a team member does not complete their assigned work, they will receive a grade of **zero** for this stage.

However, the team's primary goal is to ensure the overall system remains functional. In the event that a team member fails to deliver their portion, the rest of the team is expected to redistribute tasks or create alternative solutions to address the missing functionality.

It is critical to document such situations clearly. If a team member does not fulfill their responsibilities, this must be noted in the peer grading form, detailing how their absence impacted the project and how the team mitigated it. While teamwork is encouraged, accountability is paramount—no free rides!

Submission:
a) Submit in Canvas the GitHub **link** to access the corresponding folder (be aware of modification date!)
b) Submit in Canvas your peer grading form.

Due date: Sunday, April 12 (10:00 p.m.) [tentative]

Stage 4 (100 points)

In this stage, you need to do the following:

a) Implementation of the system as a ***GUI*** application (**60 points total**)
   i) Your system must have a **main file** (Main.java) to run and separate Java files for each class. (**10 points**)
   ii) You must use inheritance for your custom classes (not default inheritance from the GUI interfaces). (**5 points**)
   iii) Your **main window** must have a **menu** to navigate through the different sections. (**5 points**)

iv) Your system must have a **window** to show on screen **at least two** reports. (**5 points**)

v) The GUI interface must use **multiple windows** (e.g., open new windows or use a tabbed panel) (**5 points**)

vi) The system must allow the user to **create**, **read**, **modify**, and **delete** (CRUD) the information of the main objects. For example, client information, car information, employee information, patient information, item information). (**15 points**)

vii) The system **must** use **persistent information**. This is, **Load** and **Save** information in **local files** (e.g., .csv, .txt, .dat) Do not use databases. Use text files. (**10 points**)

viii) Submit the persistent files (.csv, .txt., .dat) with information. (**5 points**)

b) The project needs to have **catalogs** of objects (e.g., a catalog of clients, a catalog of items, a catalog of sales, etc.). Therefore, a **search** method is expected (**10 points**)

c) The system **must** have an executable Jar file. Upload to **GitHub** your source files and the **Jar** file into the corresponding folder for part 3 (**5 points**)

d) Upload to GitHub a **readme.txt** file with the instructions on how to compile and how to run your program (this is important!) (**3 points**)

e) Upload to GitHub a **PDF** file with the **modifications** to your basic UML diagrams representing your program's classes (i.e., attributes, methods) and their relationships (**5 points**)

f) Your project's folder must contain a **TXT file per team member** with the job done by that team member. Everybody needs to work on the GitHub repository. I'll check the commits. (**2 points**)

g) Upload to GitHub a **PDF** document a brief **user guide** on how to use your system and the most important sections (use screenshots of your system running and descriptions of each image) (**5 points**)

h) One team member needs to be the project leader for this stage (different from the previous stage) and needs to meet me before the due date for reviewing the work done.

The meeting needs to be done **before** the due date to present the progress of the work. **No meetings will be allowed during final's week.** If the team leader does not meet with me, the whole team will get the deduction. The team members must hold the team leader accountable. The team leader will receive an additional deduction as they have the responsibility to represent the team.(**10** points).

Important: Each team member is fully responsible for completing the implementation of the classes they selected in Stage 1. If a team member does not complete their assigned work, they will receive a grade of **zero** for this stage.

However, the team's primary goal is to ensure the overall system remains functional. In the event that a team member fails to deliver their portion, the rest of the team is expected to redistribute tasks or create alternative solutions to address the missing functionality.

It is critical to document such situations clearly. If a team member does not fulfill their responsibilities, this must be noted in the peer grading form, detailing how their absence impacted the project and how the team mitigated it. While teamwork is encouraged, accountability is paramount—no free rides!

Submission:

1) Submit in Canvas the GitHub **link** to access the corresponding folder (be aware of modification date!)
2) Submit in Canvas your peer grading form.

Each team will have a **maximum of 10 minutes** on the final exam date to present their project. At least **one** of the team members needs to attend via MS Teams and present the general functionality. **If a team does not present the project, no points will be granted for the last stage of the project, and the grade for this stage will be zero. It has happened! Don't risk your grade.**

Due date and Project presentation: Friday May 8 Time: **8:15 am to 10:15 am** (Mountain Time)
Submission of Stage 4: Friday May 8 Time: **10:15 AM** (Mountain Time)

Remote students: make your arrangements (e.g., if you work) to present your project.

Peer-grading

- Each team member **must** grade the effort of the others.
- Your grade will be determined by how your teammates perceived your effort and participation in the project. There are no free rides!
- Read the document "Project Evaluation Form.doc" for more details.

I will give **5 extra points** for the last stage to the team that presents the best overall project.

To have an idea of what your system is expected to do, and the documentation required, please see the examples I have shared on Canvas.

Tip: Start looking for a team after this class! Do not wait! Once you have a team, please update the "Teams Information" page.

Please, do not assume things. Ask me if you have any questions.